RAMDEOBABA UNIVERSITY, NAGPUR

Department of Computer Science and Engineering

B.Tech. CSE (Artificial Intelligence and Data Science)

A Project Report on

BigBasket AI — AI-Based Product Price Prediction and Market Analytics System

Submitted for SI Lab, Semester V

Submitted by

Ayush Akare

Roll Number: 44 | Section: B2

Academic Year 2026–27

Table of Contents

1. Project Objective **1**
2. Introduction **1**
2.1 Problem Statement 1
2.2 Scope 2
3. Project Methodology **2**
3.1 Dataset Loading and Exploration 2
3.2 Data Preprocessing 4
3.3 Feature Extraction and Selection 4
3.4 Actual Models Implemented 4
3.5 Performance Evaluation and Model Comparison 5
3.6 Best Model Selection 6
4. Project Implementation **6**
4.1 Tools and Technologies 6
4.2 Module-wise Implementation 6
4.3 GUI Implementation 7
5. Complete Code / Source Code Reference **7**
6. Results and Discussion **9**
6.1 Application Screenshots 9
6.2 Model Outputs and Error Analysis 13
6.3 Discussion 14
7. Conclusion **14**
8. GitHub Repository Link **14**
9. References **15**

1. Project Objective

The objective of this project is to build **BigBasket AI**, a Python application that analyses a BigBasket product catalogue and predicts a product's selling price from its Maximum Retail Price (MRP) and catalogue labels. The project turns an exploratory SI notebook into a modular, tested Streamlit application backed by a scikit-learn regression pipeline. Its specific objectives, all implemented in the submitted code, are:

1. Load, validate and clean the 1,249-product BigBasket dataset with reusable, reproducible rules.
2. Analyse selling prices, MRPs, discounts and ratings across categories, brands and product types.
3. Predict the selling price (`Price`) from MRP, Category, Subcategory, Brand and Product_Type without target leakage.
4. Compare Gradient Boosting with Linear Regression and Random Forest using MAE, RMSE and R^2 on a held-out test set and 5-fold cross-validation.
5. Provide an eight-page interactive GUI for exploration, comparison, price prediction and model inspection.

2. Introduction

Online grocery platforms list thousands of packaged products. Each carries a printed MRP, and the platform's selling price is the MRP minus a discount that varies from product to product. In the project dataset, 583 of 1,249 products (46.7%) sell at full MRP, while others are discounted by up to 79.24%. Understanding what a typical selling price looks like, and how discounts differ across the catalogue, is useful for buyers, sellers and analysts, but cannot be done reliably by hand.

The project was developed in two stages. An exploratory notebook (`notebook/SI_Project.ipynb`, executed in Google Colab) cleaned the data, explored it and trained a Gradient Boosting Regressor. The notebook logic was then refactored into the application `BigBasket_AI/`, which reproduces the notebook model exactly and adds baseline models, cross-validation, prediction ranges, input checks and a Streamlit interface.

2.1 Problem Statement

Given a product's MRP and its catalogue labels (Category, Subcategory, Brand and Product Type), estimate its selling price on BigBasket with measured accuracy, and provide an interactive tool to explore how prices, discounts and ratings vary across the catalogue. The solution must avoid target leakage (the dataset contains the price-derived columns `Price_Segment` and `Discount`), must not fail on unseen labels (447 of 689 brands occur only once), and must report accuracy on products held out from training.

2.2 Scope

Table 2.1: *Project scope*

In scope (implemented and verified)	Out of scope
Cleaning and validating the supplied BigBasket CSV (three column-naming schemes accepted)	Live scraping (the notebook records HTTP 403 from BigBasket)
Descriptive analytics, IQR outlier rules and rule-based insights	Text features from product names or descriptions
Regression of selling price; three models compared on one split plus 5-fold CV	Hyper-parameter search; predicting ratings or demand
Quantile-based prediction range with measured test coverage	Statistical guarantees for the range
Eight-page Streamlit GUI, model persistence, automated smoke test	Public deployment, user accounts, external AI services

3. Project Methodology

The project follows the supervised-learning workflow in Figure 3.1. Each stage corresponds to identifiable code in the application.

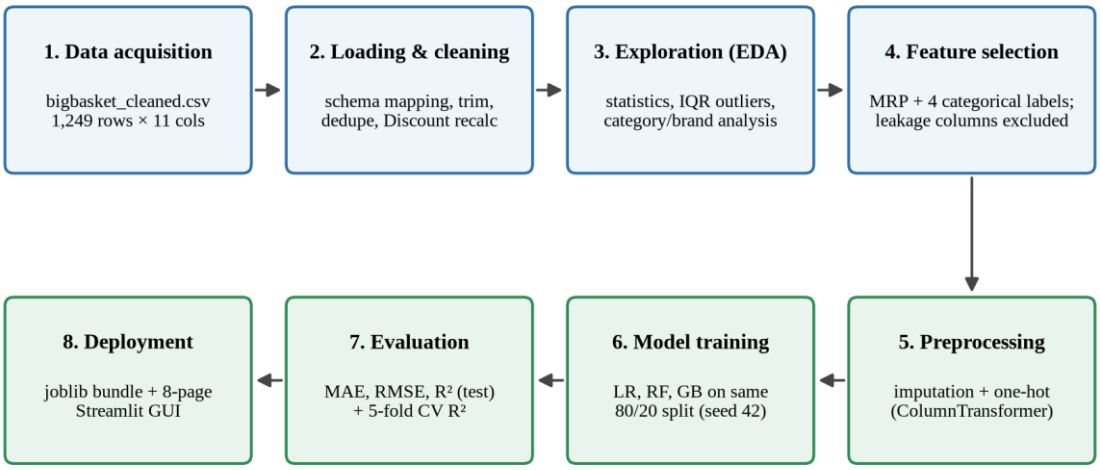


Figure 3.1: *Project workflow from data acquisition to the deployed application*

3.1 Dataset Loading and Exploration

The dataset is `data/bigbasket_cleaned.csv`, the cleaned output of the SI notebook. According to the notebook and `data/README.md`, it is a 1,249-record sample of the public Kaggle dataset "BigBasket Entire Product List (~28K datapoints)" [1]; the original sample file is not included in the ZIP, so the sampling step itself cannot be re-verified. `find_dataset_file()` locates the CSV in `data/`, and `load_dataset()` reads it (retrying with Latin-1 encoding if needed), cleans it and records a SHA-256 fingerprint used to detect when the saved model is out of date.

The file has **1,249 rows and 11 columns**, with **0 duplicate rows**. Only **Rating** has missing values: **394 products (31.55%)** are unrated. The catalogue covers 11 categories, 83 subcategories, 689 brands and 289 product types. Table 3.1 lists the columns and their roles.

Table 3.1: *Dataset columns (measured on bigbasket_cleaned.csv)*

Column	Type	Description	Role in model
Item_ID	int64	Source index (unique)	Not used
Product	object	Product title (1,237 unique)	Not used (search only)
Category / Subcategory	object	11 / 83 values	Features
Brand / Product_Type	object	689 / 289 values	Features
MRP	float64	Printed maximum retail price (₹)	Feature
Price	float64	Selling price on BigBasket (₹)	Target
Rating	float64	1–5; 855 non-null	Analytics only
Price_Segment, Discount	object, float64	Low/Medium/High band; (MRP − Price)/MRP × 100	Excluded (leakage)

Table 3.2: *Descriptive statistics of the numeric columns*

Column	Count	Mean	Std	Min	Median	Max
Price (₹)	1,249	334.31	504.20	5.00	195.30	6,700.00
MRP (₹)	1,249	394.14	608.63	5.00	225.00	7,875.00
Discount (%)	1,249	11.10	14.42	0.00	5.00	79.24
Rating	855	3.93	0.77	1.00	4.10	5.00

Prices are strongly right-skewed (mean ₹334.31 versus median ₹195.30). Beauty & Hygiene is the largest category (394 products, 31.5%); Kitchen Garden & Pets has the highest average price (₹531.74) and the deepest average discount (19.67%); Fruits & Vegetables is the cheapest (₹32.88). The Pearson correlation between MRP and selling price is **0.962**, while rating and price are essentially unrelated (Spearman ρ = −0.069). The IQR rule flags 112 price outliers, which were kept because they are genuine premium products. Figure 3.2 shows the distributions.

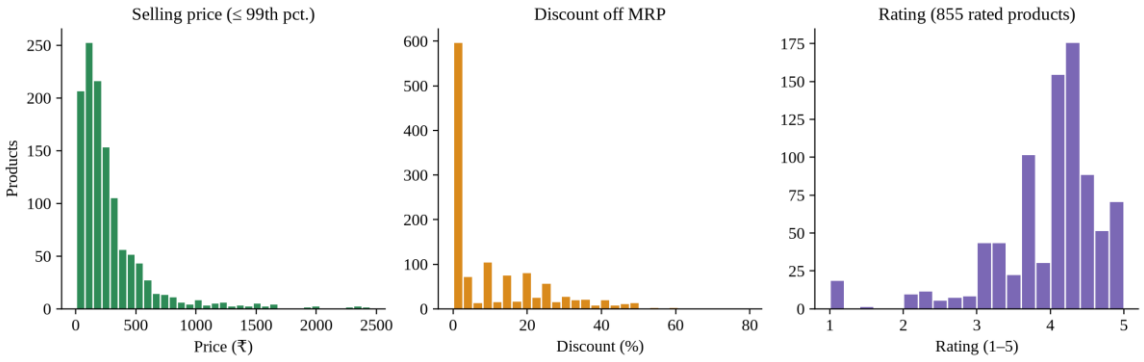


Figure 3.2: *Distributions of selling price (≤ 99th percentile), discount and rating*

3.2 Data Preprocessing

`clean_dataset()` (`src/data_loader.py`) reproduces the notebook's cleaning: it standardises column names, converts numeric columns, trims text, drops rows without a product name and exact duplicates, fills missing labels with "Unknown", sets negative prices and ratings outside 0–5 to missing, and recalculates `Discount = (MRP − Price) / MRP × 100` clipped to 0–100. On the project file these rules remove **0 rows**, because the file is already clean; they matter because the app also accepts the raw notebook file and the original Kaggle file.

`build_preprocessor()` (`src/preprocessing.py`) creates a `ColumnTransformer` [4]: MRP passes through median imputation, and the four categorical columns pass through most-frequent imputation and `OneHotEncoder(handle_unknown="ignore")`, so an unseen brand or type never causes an error. `split_data()` applies `train_test_split(test_size=0.20, random_state=42)`, giving **999 training and 250 test products**, identical to the notebook.

3.3 Feature Extraction and Selection

Features were selected explicitly from domain knowledge (Table 3.3). After one-hot encoding on the training rows the model receives **962 input columns** (1 MRP, 11 Category, 81 Subcategory, 601 Brand, 268 Product_Type). No text features are extracted from product titles.

Table 3.3: *Feature selection decisions*

Column(s)	Decision	Reason
MRP	Numeric feature	Known before pricing; strongest predictor (r = 0.962)
Category, Subcategory, Brand, Product_Type	Categorical features	Capture category, brand and type discount habits
Price_Segment, Discount	Excluded	Derived from the selling price (target leakage)
Rating	Not used	31.55% missing; not part of the notebook design
Item_ID, Product	Not used	Identifier and free text

The fitted Gradient Boosting model confirms the dominance of MRP: summed per original feature, the importances are MRP 0.9842, Brand 0.0093, Product_Type 0.0037, Subcategory 0.0026 and Category 0.0002.

3.4 Actual Models Implemented

`candidate_models()` (`src/model.py`) wraps three regressors in the same preprocessing pipeline (Table 3.4). `train_models()` fits each on the 999 training rows, computes test and training metrics, runs shuffled 5-fold cross-validation on all 1,249 rows with a fresh pipeline clone, then fits two quantile models for the prediction range. All results are saved in `models/price_model_bundle.joblib`, which was trained on 2026-10-24 with scikit-learn 1.6.1 in 6.5 s.

Table 3.4: *Models trained (explicit hyper-parameters; others are scikit-learn defaults)*

Model	Role	Hyper-parameters
Linear Regression	Baseline	defaults
Random Forest Regressor [6]	Baseline	n_estimators=200, random_state=42, n_jobs=−1
Gradient Boosting Regressor [5][10]	Deployed model	n_estimators=150, learning_rate=0.05, max_depth=2, random_state=42
Gradient Boosting, quantile loss	Prediction range	alpha=0.10 and alpha=0.90, same settings as above

3.5 Performance Evaluation and Model Comparison

The target is continuous, so this is a regression task evaluated with MAE, RMSE and R^2 [3]; classification metrics (accuracy, precision, recall, F1, confusion matrix) do not apply. With actual price y, prediction $\hat{y}$ and mean $\bar{y}$ over n test products:

$$MAE = (1/n)\,\Sigma\,|y - \hat{y}| \qquad RMSE = \sqrt{[(1/n)\,\Sigma\,(y - \hat{y})^2]} \qquad R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$$

Table 3.5: *Model comparison: 250-product test set and 5-fold cross-validation (saved bundle)*

Model	Test MAE (₹)	Test RMSE (₹)	Test R^2	Train R^2	CV R^2 (mean ± std)
Random Forest Regressor	49.82	165.47	0.838	0.987	0.877 ± 0.032
Gradient Boosting Regressor	57.43	183.70	0.800	0.980	0.889 ± 0.047
Linear Regression	70.36	189.22	0.788	0.997	0.888 ± 0.066

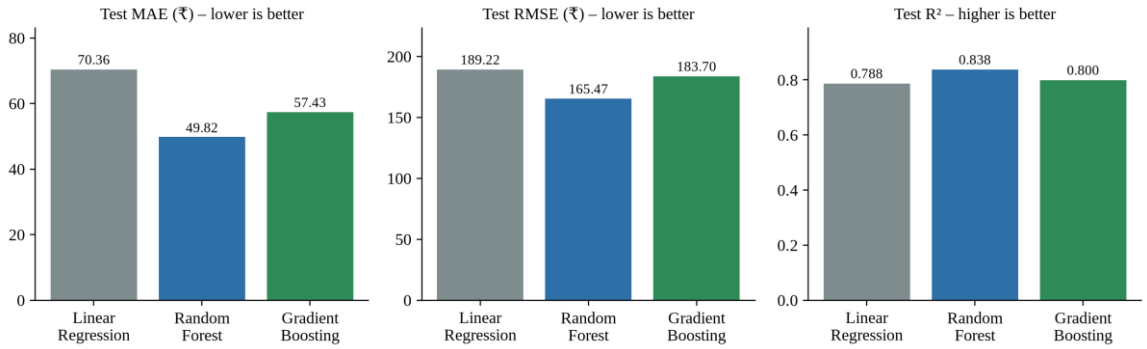


Figure 3.3: *Test MAE, RMSE and R^2 of the three models*

Random Forest is best on all three test metrics. Gradient Boosting reproduces the notebook exactly (MAE 57.434, RMSE 183.705, R^2 0.800) and has the highest mean CV R^2 (0.889), but the CV differences between models are smaller than their fold-to-fold standard deviations. All models fit the training data much better than the test data (train R^2 0.98–0.997), so the categorical labels add limited generalisable information beyond MRP. To verify the numbers, `scripts/train_model.py` was re-run for this report with the same library versions on Linux: Gradient Boosting and Linear Regression matched exactly, while Random Forest differed only in the third decimal (RMSE 165.96 vs 165.47), a normal cross-platform effect.

3.6 Best Model Selection

The code ranks models by test RMSE and reports the winner on the Model Lab page; for the saved bundle this is **Random Forest Regressor**. The application nevertheless keeps **Gradient Boosting** as its prediction model because (1) it is the model validated in the SI notebook and is reproduced exactly, (2) it has the highest mean cross-validated R^2, and (3) the same algorithm with quantile loss provides the prediction range. The application states this trade-off openly. On predictive accuracy alone the evidence favours Random Forest, and switching to it is recommended as future work (Section 7).

4. Project Implementation

4.1 Tools and Technologies

Table 4.1: *Software stack (versions from the project's .venv)*

Tool	Version	Use
Python	3.9.6	Programming language (macOS virtual environment)
Streamlit	1.50.0	Multipage web GUI, widgets, caching [7][8]
pandas / NumPy	2.3.3 / 2.0.2	Data loading, cleaning and aggregation
scikit-learn	1.6.1	Pipeline, encoders, models, metrics, cross-validation [2]
Plotly	7.1.0	Interactive charts with a custom dark template
joblib	1.5.3	Saving and loading the model bundle [9]
Google Colab (notebook only)	—	Exploration; Matplotlib, Seaborn and a Gradio product-lookup demo

4.2 Module-wise Implementation

The application contains 2,682 lines of Python, CSS and configuration, organised as an entry point, a `src/` package and one file per page in `views/` (Table 4.2).

Table 4.2: *Main source files and their responsibilities*

File	Lines	Responsibility
app.py	116	Page set-up, data/model loading, st.navigation routing, set-up and error pages
src/config.py	72	Paths, features, target, split/CV settings, GB hyper-parameters
src/data_loader.py	182	Locate, read, standardise and clean the dataset; quality report
src/preprocessing.py	64	ColumnTransformer, feature/target selection, train–test split
src/model.py	253	Train, evaluate, save, load (fingerprint + version check), predict
src/state.py	44	@st.cache_data for the dataset; @st.cache_resource for the model
src/analytics.py, insights.py	248	KPIs, filters, IQR rules, Value Score, rule-based insights
src/charts.py, ui.py, utils.py	244	Plotly theme, HTML components, formatting and escaping
views/ (8 pages)	1,143	The eight GUI pages
scripts/train_model.py, tests/smoke_test.py	115	CLI training; headless test of all pages

Prediction logic. `predict_price()` builds a one-row frame from the five inputs, takes the point estimate from the Gradient Boosting pipeline and a range from the 10th/90th-percentile models (widened if needed to contain the estimate), and returns the saving, implied discount and an "above MRP" flag. `check_inputs()` warns when a label was not seen in training, when a label combination is new, or when MRP lies outside the training range (₹5–₹7,875).

Persistence and caching. `get_or_train()` reuses the saved bundle only if its dataset fingerprint and scikit-learn version match the current ones; otherwise it retrains and saves. `src/state.py` caches the dataset and model so that all pages share them. **Robustness:** `app.py` shows an upload page if no dataset exists, separate pages for unreadable data or failed training, and catches page-level exceptions.

4.3 GUI Implementation

The GUI is a Streamlit multipage app with a custom dark theme (`.streamlit/config.toml`, `assets/style.css`). The sidebar shows navigation and a status block (dataset, product count, model test R^2). Table 4.3 lists the verified functions of each page; screenshots are presented in Section 6.1.

Table 4.3: *Streamlit pages and their verified functionality*

Page	Implemented functionality
Command Center	Seven KPI cards; data-generated market snapshot; charts of top categories, price, discount and rating distributions
Product Intelligence	Search; cascading category/subcategory/brand/type filters; price, discount and rating sliders; 6 sort orders; table or card view; CSV download; product detail with peer comparison and model estimate
AI Price Predictor	MRP and label inputs (or a held-out test product); new-value entry; price tag with range and coverage; actual-vs-estimate error; reliability warnings; MRP sensitivity curve
Market Analytics	Filters; tabs for Pricing (MRP vs price, category/brand/type charts), Discounts, Ratings and Top products
Product Comparison	Compare 2–4 products; quick-pick of three best-rated in a subcategory; side-by-side table and charts; adjustable Value Score with conclusion
AI Insights	Ten rule-based insight cards; category scorecard; price anomalies; deep discounts; largest test-set errors
Model Lab	Test metrics, model card, data summary, model comparison, actual-vs-predicted and residual charts, feature importance, range coverage, CSV exports, retrain button
About Project	Problem, objectives, dataset, method, technology versions, limitations, future scope, team

5. Complete Code / Source Code Reference

No public repository or shareable Colab link exists for this project (Section 8). The complete, runnable source code is contained in the submitted `BigBasket_AI.zip`, whose structure and line counts were verified for this report (Table 4.2). The application runs with:

```
cd BigBasket_AI
python -m venv .venv && source .venv/bin/activate    # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python scripts/train_model.py     # optional: retrain and print the evaluation table
python tests/smoke_test.py        # optional: headless test of every page
streamlit run app.py              # opens http://localhost:8501
```

The following excerpts, copied from the source files, show the core of the machine-learning implementation.

Feature definition and leakage prevention (`src/config.py`):

```
TARGET = "Price"
NUMERIC_FEATURES = ["MRP"]
CATEGORICAL_FEATURES = ["Category", "Subcategory", "Brand", "Product_Type"]
FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES
# Price_Segment is NOT a feature: it may be derived from Price (target leakage).
LEAKAGE_COLUMNS = ["Price_Segment", "Discount"]
TEST_SIZE = 0.20
RANDOM_STATE = 42
CV_FOLDS = 5
GB_PARAMS = dict(n_estimators=150, learning_rate=0.05, max_depth=2,
random_state=RANDOM_STATE)
```

Preprocessing pipeline (`src/preprocessing.py`):

```
def build_preprocessor() -> ColumnTransformer:
    numeric_pipeline = Pipeline([("imputer", SimpleImputer(strategy="median"))])
    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", make_one_hot_encoder()),   # OneHotEncoder(handle_unknown="ignore")
    ])
    return ColumnTransformer([
        ("numeric", numeric_pipeline, NUMERIC_FEATURES),
        ("categorical", categorical_pipeline, CATEGORICAL_FEATURES),
    ])
```

Candidate models and evaluation loop (`src/model.py`, abridged):

```
def candidate_models() -> dict[str, Pipeline]:
    return {
        "Linear Regression": make_pipeline(LinearRegression()),
        "Random Forest Regressor": make_pipeline(
            RandomForestRegressor(n_estimators=200, random_state=RANDOM_STATE, n_jobs=-
1)),
        MAIN_MODEL_NAME: make_pipeline(GradientBoostingRegressor(**GB_PARAMS)),
    }

for i, (name, pipe) in enumerate(models.items()):
    pipe.fit(X_train, y_train)
    test_m = regression_metrics(y_test, pipe.predict(X_test))
    train_m = regression_metrics(y_train, pipe.predict(X_train))
    cv_scores = cross_validate(clone(pipe), X, y, cv=cv,
                               scoring=("r2", "neg_mean_absolute_error"))
comparison = pd.DataFrame(rows).sort_values("RMSE").reset_index(drop=True)
best_by_rmse = comparison.iloc[0]["Model"]
```

Prediction with a range (`src/model.py`, abridged):

```
frame = pd.DataFrame([row])[FEATURES]
point = float(bundle["pipeline"].predict(frame)[0])
low = float(bundle["q_low"].predict(frame)[0])
high = float(bundle["q_high"].predict(frame)[0])
low, high = min(low, high), max(low, high)
widened = point < low or point > high
low, high = min(low, point), max(high, point)
return {"prediction": point, "low": max(low, 0.0), "high": high,
        "saving": mrp - point, "discount_pct": (mrp - point) / mrp * 100,
        "above_mrp": point > mrp, "notes": check_inputs(bundle, row), ...}
```

6. Results and Discussion

6.1 Application Screenshots

All screenshots are real captures of the submitted, unmodified application running on the submitted dataset and saved model. They were captured for this report with a headless Chromium browser at 1,440 px width in an environment pinned to the project's library versions; system fonts replaced the theme's web fonts.

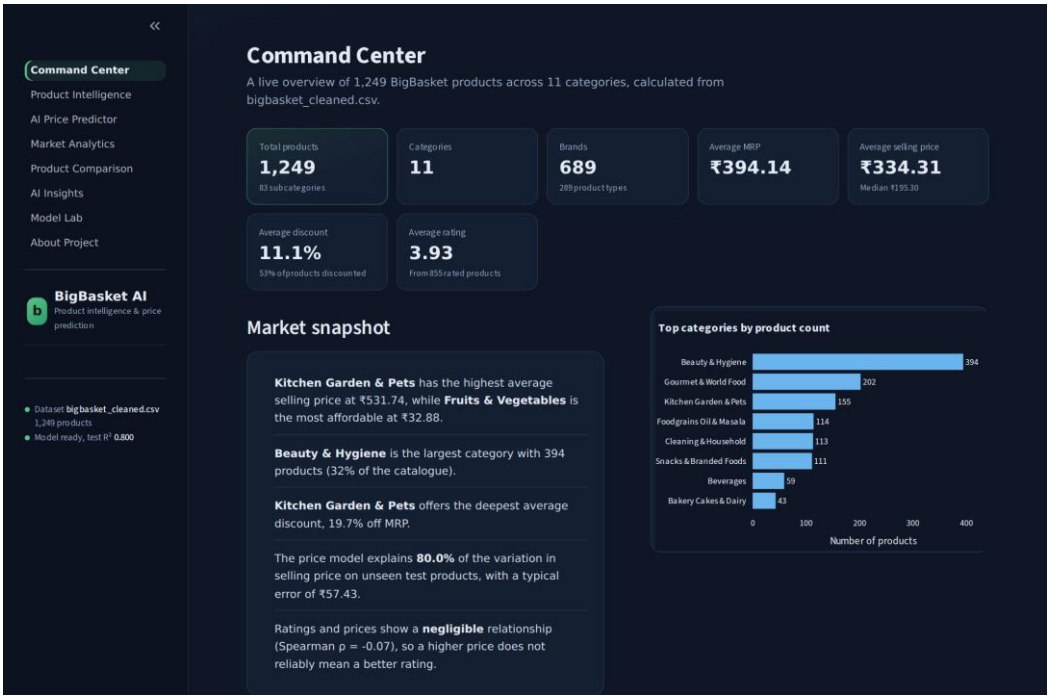


Figure 6.1: *Command Center: KPI cards, data-generated market snapshot and top categories*

The Command Center (Figure 6.1) reports 1,249 products, 11 categories, 689 brands, average MRP ₹394.14, average selling price ₹334.31, average discount 11.1% and average rating 3.93. Its snapshot sentences, including the model's 80.0% explained variance and ₹57.43 typical error, are generated from the data at run time.

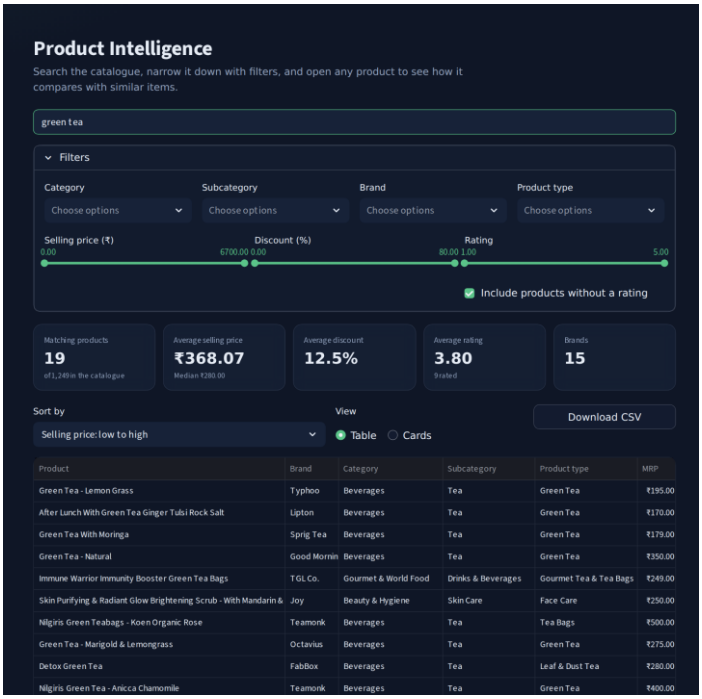


Figure 6.2: *Product Intelligence: search for "green tea" with filters, summary KPIs and results table*

In Figure 6.2 the search "green tea" returns 19 products from 15 brands (average price ₹368.07), which can be sorted, viewed as cards or downloaded as CSV.

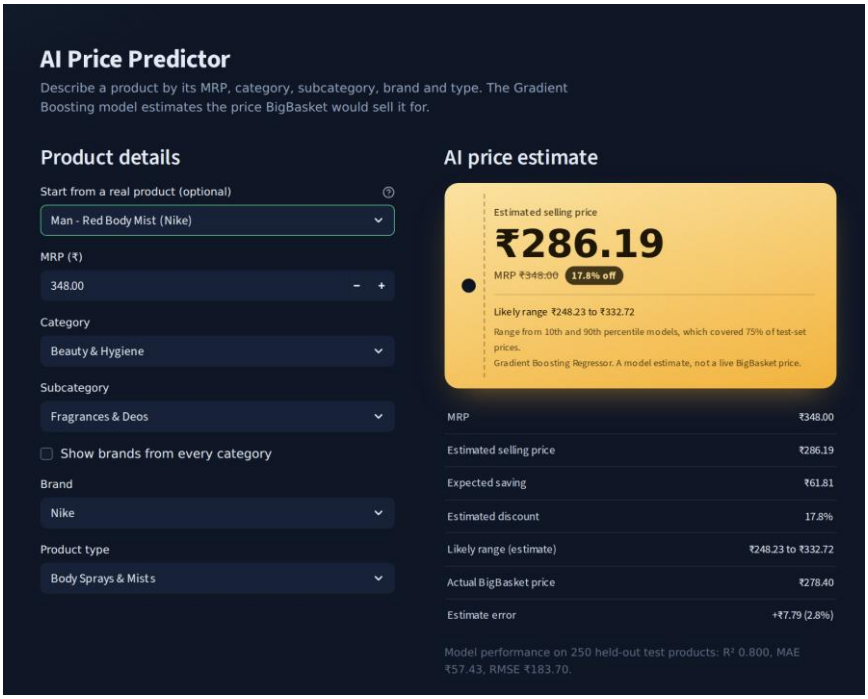


Figure 6.3: *AI Price Predictor: held-out test product and the resulting price estimate*

Figure 6.3 shows a real prediction for a held-out product, *Man – Red Body Mist (Nike)*, MRP ₹348: the model estimates **₹286.19** (17.8% off, likely range ₹248.23–₹332.72) against an actual price of ₹278.40, an error of +₹7.79 (2.8%).



Figure 6.4: *Market Analytics, Pricing tab: filters and MRP versus selling price by category*

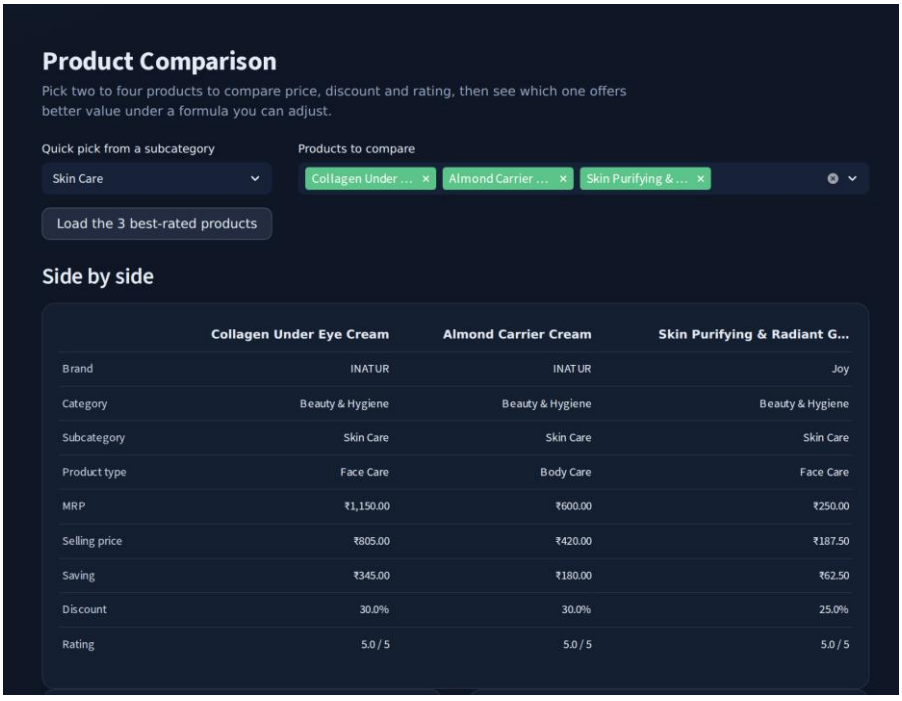


Figure 6.5: *Product Comparison: quick-pick of three Skin Care products, compared side by side*

Market Analytics (Figure 6.4) shows that almost all products lie on or below the Price = MRP line (Pearson r = 0.962). Product Comparison (Figure 6.5) loads the three best-rated Skin Care products; its Value Score (50% discount, 50% rating by default) rates the first two as tied at 68.9.

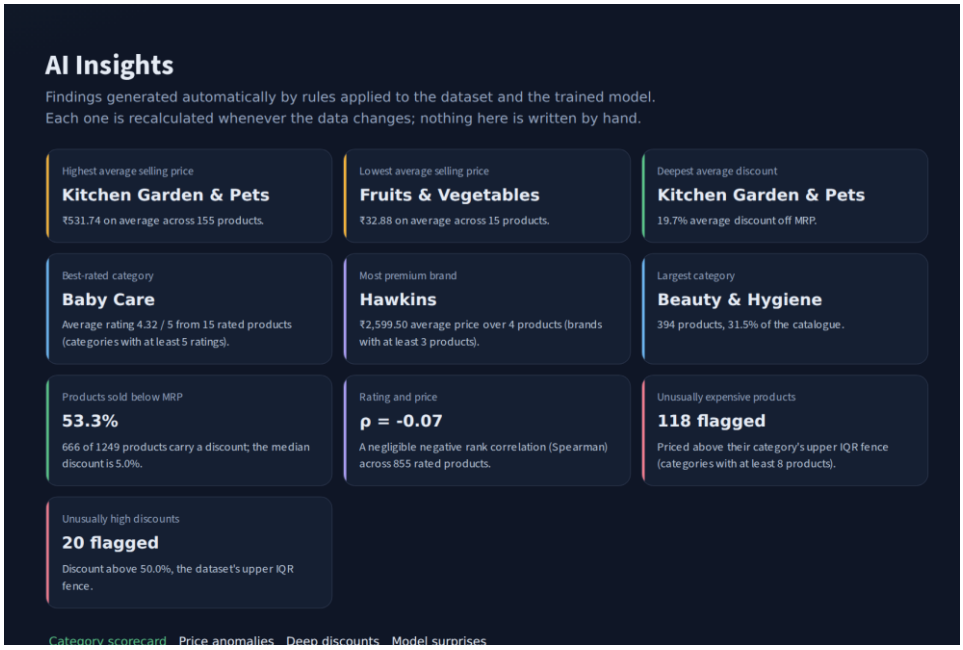


Figure 6.6: *AI Insights: rule-based insight cards computed from the dataset*

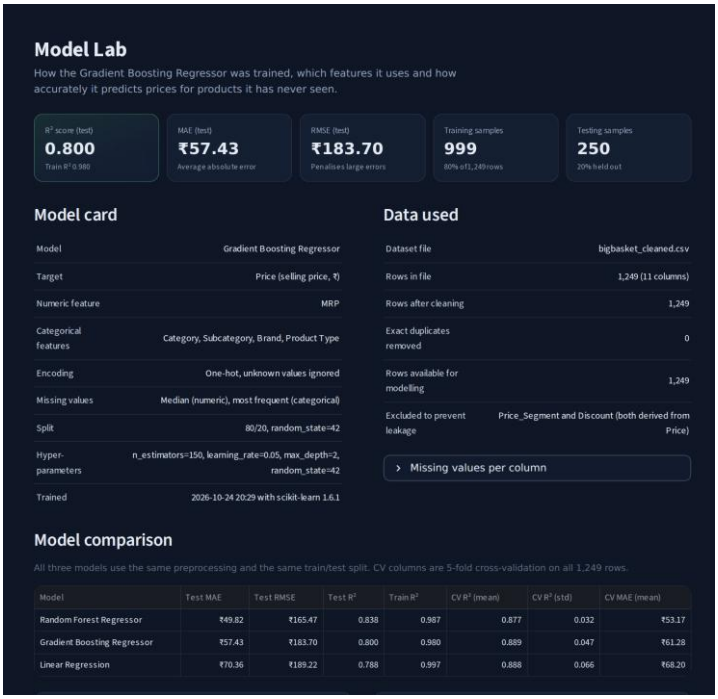


Figure 6.7: *Model Lab: test metrics, model card and data summary*

AI Insights (Figure 6.6) shows ten rule-based findings: for example 53.3% of products are sold below MRP, Hawkins is the most premium brand (₹2,599.50 average over 4 products), and 118 products are priced above their category's upper IQR fence. The Model Lab (Figure 6.7) displays the saved model's test R^2 0.800, MAE ₹57.43 and RMSE ₹183.70, with the model card and data summary; further down the same page it shows the comparison table, diagnostic charts and export/retrain controls.

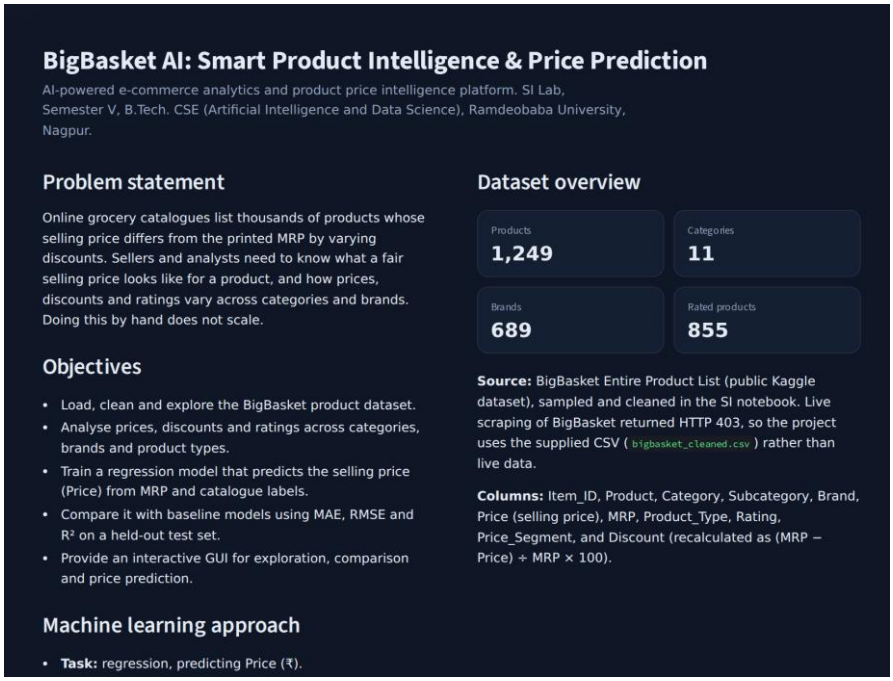


Figure 6.8: *About Project: problem statement, objectives, dataset overview and approach*

The About Project page (Figure 6.8) summarises the problem, objectives, dataset, machine-learning approach with live test metrics, technology versions, limitations, future scope and the team details read from `src/project_info.py`.

6.2 Model Outputs and Error Analysis

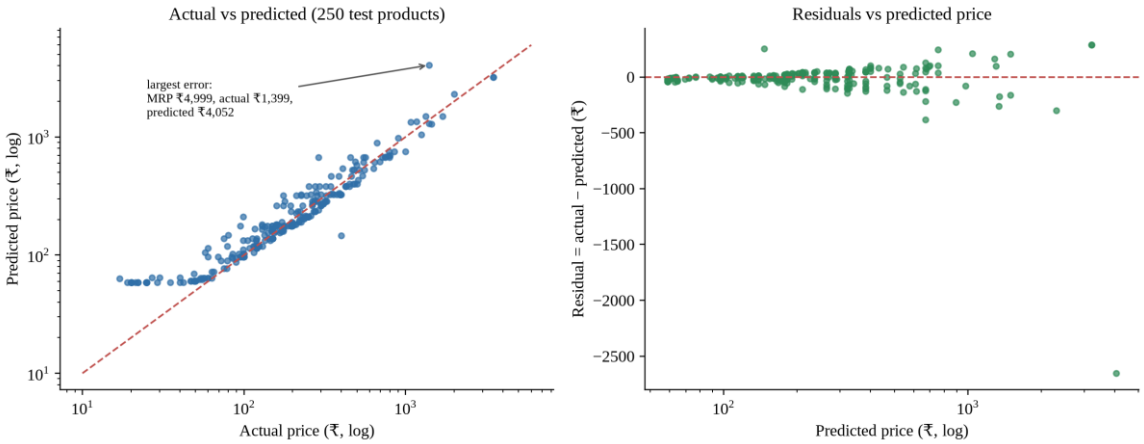


Figure 6.9: *Gradient Boosting on the 250 test products: actual vs predicted (left) and residuals (right)*

Most predictions in Figure 6.9 lie close to the diagonal across three orders of magnitude. The median absolute error is ₹31.93; 33.6% of predictions are within 10% of the actual price and 69.6% within 20%. The prediction range covered 74.8% of test prices (target 80%) with a median width of ₹90.78. One product dominates the squared error: a Nirlon cookware set with MRP ₹4,999 sold at ₹1,399 (72% off) was predicted at ₹4,051.85. Recomputing without that single product gives RMSE ₹74.96 and R^2 0.966. In 49 of the 250 test cases the raw prediction exceeds MRP; the predictor page flags and explains this.

To judge what the model adds beyond MRP, two simple rules were evaluated on the same test set for this report (Table 6.1; not part of the application).

Table 6.1: *Models compared with rule-based baselines on the same 250 test products*

Predictor	MAE (₹)	RMSE (₹)	R^2
Price = MRP (no discount)	61.29	254.03	0.617
Price = MRP × (1 − 0.1082), mean training discount	56.89	211.81	0.734
Gradient Boosting Regressor (deployed)	57.43	183.70	0.800
Random Forest Regressor	49.82	165.47	0.838

6.3 Discussion

The results show that selling price is largely "MRP minus a discount". MRP carries 98.4% of the Gradient Boosting importance, and a constant-discount rule already matches Gradient Boosting on MAE. The learned models add value mainly on expensive items, where they reduce large errors (RMSE ₹183.70 for Gradient Boosting and ₹165.47 for Random Forest, versus ₹211.81 for the rule). Errors come from unusual promotions, sparse labels (90 of 250 test products have a brand unseen in training) and high percentage errors on very cheap items. The market analysis shows selective discounting (median 5%; Kitchen Garden & Pets and Fruits & Vegetables discount most) and no meaningful link between price and rating.

The application was also verified functionally: `tests/smoke_test.py` was executed against the submitted code and passed all 11 checks. These covered all eight pages, an unseen brand with an out-of-range MRP (warning shown, no crash), a search with no matches, and the comparison quick-pick.

7. Conclusion

BigBasket AI meets its objectives. It cleans and analyses a 1,249-product BigBasket sample and predicts selling price with a leakage-free scikit-learn pipeline. The deployed Gradient Boosting model reproduces the notebook exactly (test MAE ₹57.43, RMSE ₹183.70, R^2 0.800) and has the best cross-validated R^2 (0.889). The Random Forest baseline is the most accurate on the test set (R^2 0.838), and both the application and this report state this openly. Eight tested Streamlit pages make the analysis, predictions and model evaluation accessible to non-programmers.

Limitations: a static sample rather than live data; MRP dominates the prediction; sparse brand labels; a third of ratings missing; predictions can exceed MRP; untuned hyper-parameters. **Future scope:** deploy Random Forest or a tuned ensemble; predict the discount and cap estimates at MRP; add text features from product titles; train on the full ~28K-product list; deploy on Streamlit Community Cloud.

8. GitHub Repository Link

Repository URL: https://github.com/ayush10-design/BigBasket-AI?utm_source=chatgpt.com

9. References

[1] surajjha101 (Kaggle user), "BigBasket Entire Product List (~28K datapoints)," Kaggle. https://www.kaggle.com/datasets/surajjha101/bigbasket-entire-product-list-28k-datapoints (accessed 26 Oct. 2026).

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[3] scikit-learn developers, "Metrics and scoring: regression metrics," User Guide. https://scikit-learn.org/stable/modules/model_evaluation.html

[4] scikit-learn developers, "Pipelines and composite estimators," User Guide. https://scikit-learn.org/stable/modules/compose.html

[5] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[6] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[7] Streamlit, "st.navigation," documentation. https://docs.streamlit.io/develop/api-reference/navigation/st.navigation

[8] Streamlit, "Caching overview," documentation. https://docs.streamlit.io/develop/concepts/architecture/caching

[9] joblib developers, "Persistence," documentation. https://joblib.readthedocs.io/en/stable/persistence.html

[10] scikit-learn developers, "GradientBoostingRegressor," API reference. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html